Multi-Context Execution on a RISC-V Core for Mixed-Criticality Systems

Leonardo Fazzini*‡, Giacomo Valente*§, Fabio Federici†¶, Matthew P. Corbett†‖ and Tania Di Mascio***
* University of L'Aquila † Collins Aerospace
§ leonardo.fazzini1@graduate.univaq.it § giacomo.valente@univaq.it ¶ fabio.federici@collins.com
‖ matthew.corbett@collins.com ** tania.dimascio@univaq.it

2025 IEEE European Test Symposium (ETS) | 979-8-3315-9450-3/25/$31.00 ©2025 IEEE | DOI: 10.1109/ETS63895.2025.11049618

***Abstract*—Mixed-criticality systems integrating software modules with varying levels of criticality require robust isolation to ensure safety and dependability. High-end and mid-end architectures achieve this through strict time and space partitioning supported by multiple privilege levels and memory management units. However, low-end processors, limited to two privilege levels, face challenges in implementing efficient time and space partitioning without compromising system composability. This paper introduces a novel multi-context environment for low-end RISC-V processors, leveraging a lightweight hardware extension and enabling efficient time and space partitioning. The proposed solution not only ensures robust isolation and system composability but also provides flexibility to balance hardware and software overhead, advancing the state of the art in the dependability for resource-constrained systems.**

***Index Terms*—Partitioning; Mixed-Criticality; Composability.**

I. Introduction

Dependable mixed-criticality systems have garnered significant attention in domains traditionally focused on dependability, such as automotive and aerospace, driven by the need to reduce size, weight, power, and cost [1]. These systems integrate multiple SW-modules with varying levels of criticality on the same computing platform. Prior to commercialization, these systems typically undergo a rigorous certification process. As evolving requirements drive the need for continuous system upgrades, integrating and modifying parts of the system requires the recertification of the entire system.

To minimize the need for full system recertification, system architectures that support *composability* and *mixed-criticality* enable the integration of components without compromising the safety or availability of individual pre-certified SW-modules, limiting certification efforts to only the new configuration or changes. These features, in turn, are enabled by robust mechanisms for isolation, which prevent interference between SW-modules. To ensure isolation, strict *time and space partitioning* (TSP) of SW-modules is a fundamental capability demanded by standard architectures like ARINC-653 [2]. TSP can be implemented using various techniques [3], [4]. High-end and mid-end processors, such as those used in smartphones, support TSP through virtualization by utilizing multiple privilege levels and a Memory Management Unit (MMU) [5] to segregate the different SW-modules in isolated application partitions. A widely adopted approach in low-end processors, which usually do not support virtual memory, combines multiple privilege levels with a Memory Protection Unit (MPU) [5]. The MPU enforces spatial isolation by restricting memory access to defined regions for each partition. SW-modules at the highest privilege level, referred to as the *manager*, have unrestricted HW access and configure the MPU. Privilege level restrictions ensure that only higher-privilege SW can modify the MPU settings, maintaining the integrity of TSP.

Embedded and industrial systems can rely on low-end processors that support only two privilege levels [6], [7] and an MPU. Having two privilege levels presents a notable challenge: all partitions must operate at the lowest privilege level, relying entirely on the manager for tasks such as interrupt handling and core management. For systems with only two privilege levels, upgrading a SW-module that, for example, requires access to a new peripheral that makes use of an interrupt may require adding new code to the manager. This modification can increase the complexity of the manager and potentially introduce new certification challenges.

Among low-end processors, ARM-based ones addressed this limitation through ARM TrustZone-M [8], an HW-based solution that extends two-privilege-level processors by creating multiple isolated environments, referred to as contexts, with asymmetric privileges. This division effectively emulates more than two privilege levels and supports the creation of multiple partitions inside the unprivileged context. In recent years, due to its higher cost efficiency, greater modularity, and increased flexibility, RISC-V architectures [9] have been considered as alternatives to ARM-based ones in embedded systems. Although various solutions have been proposed [10]–[13], RISC-V currently lacks a standardized framework for implementing HW-enforced contexts with asymmetric privilege levels for low-end microcontrollers.

The objective of this paper is to propose a multi-context environment for low-end RISC-V devices with two privilege levels. Building on a similar concept already available in state-of-the-art solutions [10], [11], our proposal enables the creation of two contexts. Unlike existing RISC-V approaches, the proposed solution introduces a lightweight HW extension that ensures the following features:

(i) Asymmetric privilege contexts, allowing a primary context to manage secondary SW tasks and effectively exposing more than two privilege levels;
(ii) Effective TSP and dynamic runtime resource allocation across partitions;

979-8-3315-9450-3/25/$31.00 ©2025 IEEE

Authorized licensed use limited to: University of L'Aquila. Downloaded on October 24,2025 at 10:25:44 UTC from IEEE Xplore. Restrictions apply.

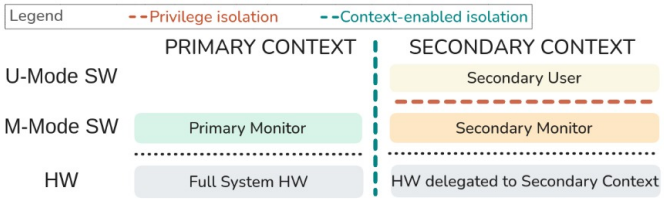

Fig. 1. Proposed Context division.

(iii) Configurable trade-offs between HW cost and SW overhead, while maintaining strong isolation guarantees.

In the remainder of the paper, Sections II and III describe the proposed approach; Section IV demonstrates the effectiveness of the proposed extension; Section V reviews related work; and Section VI concludes the paper.

II. Strategy

The proposed solution is an HW extension applicable to a RISC-V processor that has two privilege levels: *Machine-Mode* (M-Mode) for full control of the HW and *User-Mode* (U-Mode) for SW-tasks without access to critical resources. The reference RISC-V enforces memory protection with *Physical Memory Protection* (PMP) and *Enhanced PMP* (ePMP) [9], and has *Control and Status Registers* (CSRs). In the rest of this paper, register names are written in italics, while individual bits are in capital letters. The proposed solution has the objective of providing two isolated *execution contexts*: the *primary context* and the *secondary context*. This division is reported in Fig. 1 with a green dashed vertical line.

The SW executing in the primary context operates in M-mode with full control of the system HW and is referred to as *primary monitor* (PM, green box in Fig. 1). On the other hand, the secondary context can execute SW at two privilege levels: a *secondary monitor* (SM, orange box in Fig. 1) in M-mode and a *secondary user* (SU, yellow box in Fig. 1) in U-mode. Differently from PM, the SM has limited M-mode capabilities: this effectively creates asymmetric privileges. Memory isolation is ensured by assigning dedicated memory regions to each context, with no shared memory. Furthermore, the PMP-based isolation between M-Mode (SM) and U-mode (SU) remains intact, preserving separation across privilege levels. The PM controls all regions, traps, and CSRs, while the SM is limited to secondary regions, delegated non-critical traps, and a minimal CSR subset. To ensure control, the PM can preempt the SM at any time, maintaining authority over critical events and system integrity.

III. The Proposed Approach

The proposed extension acts on the CSRs, PMP, and the register file. In particular, it adds seven new CSRs: `sid` (secondary interrupt delegation), `pstatus` (primary status), `pcause` (primary cause), `pepc` (primary exception program counter), `ptvec` (primary trap-vector table address), `pscratch` (primary scratch), and `sstack` (secondary stack). We also modify the `mseccfg` CSR (associated to ePMP), to which three bits are added: `SCP` (secondary context present), `PC` (primary context), and `LPC` (last primary context). Note that ePMP may or may not be implemented on the core, but `mseccfg` CSR must be present. Finally, we introduce a new bit `S` in the `pmpXcfg` CSRs. A list of the new and modified registers, along with their addresses, is provided in Table I, while their respective meanings are explained in the rest of the section. The extension can be activated with *mseccfg.SCP*. When set, it enables privilege division between PM, SM, and SU; when unset, the core functions as a standard RISC-V processor. The bit *mseccfg.SCP=1* is sticky (cannot be reset until a system reset) and is accessible by both PM and SM. The HW-controlled `mseccfg.PC` bit indicates the active context: 1 for PM execution, and 0 for SM or SU execution. It initializes to 1 at boot and resets to 1 when a trap handled by the PM occurs. Upon executing *mret*, it takes the value from `mseccfg.LPC`. SW cannot write it. `mseccfg.LPC` bit is partially HW-controlled, storing the value of `mseccfg.PC` before entering a trap handler. It can only be modified by PM.

TABLE I
Adopted registers in the proposed approach.

CSR Name	Address(es)	Fields
mseccfg	0x747	6-31: 0; 5: LPC; 4: PC; 3: SCP; 2: RLB; 1: MMWP; 0: MML
pmpcfgX	0x3A[0-F]	7: L; 6: 0; 5: S; 3-4: A; 2: X; 1: W; 0: R
sid	0x758	same as `mie`
pstatus	0x759	13-31: 0; 11-12: MPP; 9-10: 0; 8: PPIE; 7: MPIE; 4-6: 0; 3: PIE; 0-2: 0
pcause	0x75A	same as `mcause`
pepc	0x75B	same as `mepc`
ptvec	0x75C	same as `mtvec`
pscratch	0x75D	same as `mscratch`
sstack	0x75E	same as `mscratch`

Memory protection is enforced by the `pmpXcfg.S` bit, which assigns PMP regions to either the primary (`S=0`) or secondary (`S=1`) context, allowing each context to access only its own regions; standard PMP rules apply to the primary context, PMP and ePMP (if enabled) to the secondary, with PM having full control and SM restricted to configuring only secondary regions. The PMP lookup scheme differs slightly from the base usage (see [9]) and it prioritizes matches for primary regions over secondary ones. The use of some bits from `mseccfg` register is modified to correctly set the memory protection: `MML` activates ePMP controls inside secondary context (if it is unset, the PMP rules are also applied to secondary contexts), while `MMWP` and `RLB` act only inside secondary context regions. `MML, MMWP, RLB` are sticky and accessible by both contexts.

The PM handles critical interrupts and delegates non-critical ones to the SM via the `sid` register, which defines SM-manageable interrupts. The SM can only handle these when in control of the core; otherwise, the PM retains control to protect critical tasks. The PM utilizes a new set of CSRs to manage its interrupts: `ptvec`, `pstatus`, `pepc`, and `pcause`. On the other hand, the SM relies on the standard set of registers for M-mode trap-handling. While PM-specific CSRs share the same

Authorized licensed use limited to: University of L'Aquila. Downloaded on October 24,2025 at 10:25:44 UTC from IEEE Xplore. Restrictions apply.

TABLE II
IBEX CONFIGURATIONS

Configuration	Context Division	Trap Partitioning	primary Regions Priority	Stack Banking	Register File Banking
$Ibex^s$	×	×	×	×	×
$Ibex^e_{FB}$	✓	✓	✓	✓	✓
$Ibex^e_{SB}$	✓	✓	✓	✓	×
$Ibex^e_{NB}$	✓	✓	✓	×	×
$Ibex^e_{NP}$	✓	✓	×	✓	✓

TABLE III
RESOURCE UTILIZATION ON *Artix xc7a35t* OF IBEX CONFIGURATIONS

Configuration	**LUT**	**FF**	**BRAM**	**DSP**
$Ibex^s$	54.93%	13.55%	64%	11.11%
$Ibex^e_{FB}$	+3.28%	+0.49%	+0%	+0%
$Ibex^e_{SB}$	+2.99%	+0.48%	+0%	+0%
$Ibex^e_{NB}$	+2.63%	+0.32%	+0%	+0%
$Ibex^e_{NP}$	+2.70%	+0.48%	+0%	+0%

fields as their M-mode counterparts, `pstatus` has a different field structure, as shown in Table I. The `pstatus` CSR includes fields to manage PM-handled traps and interrupts (`PIE`, `MPIE`, `PPIE`, `MPP`), enabling the PM to preempt SM-delegated interrupts without loss of context. Exceptions are handled by the PM, except for *ecall*, which is managed by the SM when the secondary context controls the core. This allows SU to invoke the SM, like the reference processor with M/U-modes. Also, the extension introduces the *pcall*, an instruction that traps to an exception always handled by PM, enabling the secondary context to invoke the PM.

The PMP module offers configurable options. It supports both prioritized and non-prioritized region matching, balancing isolation strength and timing. The extension offers three stack handling modes: *Full Banking* (FB) duplicates the entire register file for fast, SW-free context switches; *Stack pointer Banking* (SB) banks only the `sp`, reducing HW cost with added SW overhead; and *No Banking* (NB) relies entirely on SW to save/restore registers.

IV. EXPERIMENTAL ACTIVITIES

We evaluated the proposed approach implemented on Ibex[1], a 32-bit RISC-V processor similar to the reference processor adopted. Implementation was carried out on an Xilinx *Artix xc7a35t*. To illustrate the flexibility of our proposed approach, we defined several Ibex core configurations, as shown in Table II. For each configuration, we evaluated the occupied HW resources (see Table III) and verified the correct operation of the extension. The standard core is denoted as $Ibex^s$, while $Ibex^e_{FB}$, $Ibex^e_{SB}$, $Ibex^e_{NB}$, and $Ibex^e_{NP}$ represent extension variants with full, stack, no register banking, and no PMP region priority, respectively. Fig. 2 shows the resource utilization (left axis) for different numbers of active *PMP* regions (x-axis) for the $Ibex^e_{FB}$ instance. It also shows the impact of the proposed extension on the core operating frequency (right axis) in $Ibex^e_{FB}$ and $Ibex^e_{NP}$. While $Ibex^e_{FB}$ (purple line) experiences a slight reduction in the maximum operating frequency, $Ibex^e_{NP}$ (red line) maintains the same maximum operating frequency as $Ibex^s$. The proposed extension was successfully verified across all core instances through comprehensive tests confirming correct PMP operation, region priorities, context switching, system call handling, and interrupt mechanisms.

[1]https://github.com/lowRISC/ibex

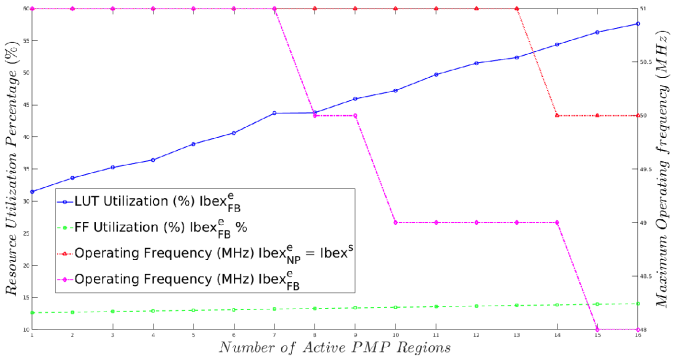

Fig. 2. Resource Utilization and maximum operating frequency as function of the number of active PMP regions.

TABLE IV
INTERRUPT LATENCIES

Instance	Non-Critical Interrupt	Critical Interrupt	Nested Interrupts		Multi-Context
			Critical	Non-Critical	
$Ibex^s$	131 cc	131 cc	131 cc	180 cc	×
$Ibex^e_{NB}$	131 cc	137 cc	137 cc	131 cc	✓
$Ibex^e_{SB}$	131 cc	131 cc	131 cc	131 cc	✓
$Ibex^e_{FB}$	131 cc	9 cc	9 cc	131 cc	✓

A. Interrupt Service Overhead

For each system instance, we measured the handling time of critical (PM-managed) and non-critical (SM-managed) interrupts, defining the total servicing time as $T_{serv} = T_{enter} + T_{ISR} + T_{exit}$, with $T_{ISR} = 0$; tasks run in SU-mode and are periodically interrupted, allowing for possible nesting. The tasks used are from EMBENCH[2]. Results in Table IV show that $Ibex^e_{NB}$ introduces minor overhead for critical interrupts due to stack pointer switching, while $Ibex^e_{SB}$ matches the timing performance of $Ibex^s$. $Ibex^e_{FB}$ achieves the highest timing performance for critical interrupts due to full register file banking. For nested interrupts, $Ibex^s$ has the lowest timing performance because non-critical interrupts must save the core state before allowing preemption, while the proposed extension improves nested interrupt handling.

B. Multi-Partition Application

The proposed approach supports multi-partition applications, as shown in Fig. 3. In this configuration, the PM alternates between two tasks running in the secondary context, assigning each task dedicated resources. A number of `pmpaddrX` and `pmpXcfg` registers equal to i define the

[2]https://github.com/embench/embench-iot

Authorized licensed use limited to: University of L'Aquila. Downloaded on October 24,2025 at 10:25:44 UTC from IEEE Xplore. Restrictions apply.

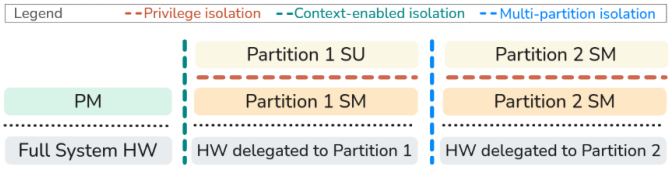

Fig. 3. Scheme of the scheduler application

memory regions assigned to the partition. The whole state of the partition is managed during context switches. In the static schedule approach, we implemented an ARINC 653-like scheduler [2] (commonly used in safety-critical avionics systems) where each task is assigned a fixed time slot for execution, and the PM alternates between partitions using a timer interrupt. For static scheduling, each partition switch requires the PM to load the resources of the partition, taking $49 + 6(i + \frac{i}{4})$ clock cycles across all configurations ($Ibex^e_{NB}$, $Ibex^e_{SB}$, and $Ibex^e_{FB}$). Since the Register File does not require saving or restoring, timing performance is consistent across these instances. In the dynamic schedule approach, each partition is assigned a priority. The lowest-priority partition executes until a higher-priority partition signals the PM that it is ready, using an interrupt. The PM then performs a partition switch by saving the state of the low-priority partition, loading the resources of the high-priority partition, and jumping to its entry point. For dynamic scheduling, partition switches require $231 + 12(i + \frac{i}{4})$ clock cycles on $Ibex^e_{SB}$. In $Ibex^e_{NB}$, an additional 6 clock cycles are needed due to the absence of optimized stack banking. However, $Ibex^e_{FB}$ cannot be used for dynamic scheduling because the PM lacks visibility into the secondary context Register File.

V. Related Works

A first alternative to our proposal is to implement a "minimal" *Supervisor mode* (*S-mode*) [9], compliant with the RISC-V ISA but without a MMU to suit small processors like the one assumed as a reference in this work. However, S-mode cannot isolate user SW-tasks without MMU, has no control over any PMP regions, and requires context-aware applications inside partitions, limiting the applicability. Similar to our work, both [10] and [11] propose architectures with two isolated contexts: a primary context with a PM in M-mode and an untrusted context with an SM and SU. However, in [11], all traps are handled by the PM, which in case of SW upgrade can impact in SW-overhead due to interaction between PM and SM; instead, [10] allows the manager to handle vectored interrupts using CLIC[3], but this requires additional HW and poses security risks because shared CSRs prevent the PM from preempting SM interrupts. Regarding CSRs, [11] restricts the SM to read-only access, lacking flexibility, while [10] grants full access, lacking protection for critical resources. To address some of the CSR-related security issues in [10], [12] introduces per-PMP-region CSR access control, allowing only the PM to access certain CSRs and specifying which CSRs are accessible by the PM. While this mitigates some security risks, it provides HW overhead with up to 64 additional CSRs (16 in the case of the Ibex implementation). [13] solves most of the issues found in the previously mentioned works by defining multiple methods for context switching, and introducing a new set of CSRs for the trusted monitor (equivalent to the PM) to manage traps. However, it introduces HW overhead, making it unsuitable for low-end devices. For instance, [13] adds 23 new CSRs, whereas our solution achieves similar functionality with only 7 CSRs, maintaining a balance between partitioning, timing performance, and HW efficiency.

[3]https://github.com/riscv/riscv-fast-interrupt

VI. Conclusions

This paper proposed a RISC-V HW extension for low-end processors, enabling two isolated contexts with asymmetric privileges. The extension was implemented on Ibex and evaluated on an Artix-7 FPGA, showing minimal SW overhead. The mechanism enables TSP, paving the way for safety-critical RISC-V adoption. Future work includes power characterization, integration with accelerators, and use in a runtime manager with dynamic partial reconfiguration for fault tolerance [14].

References

[1] C. B. Watkins and R. Walter, "Transitioning from federated avionics architectures to integrated modular avionics," in *2007 IEEE/AIAA 26th Digital Avionics Systems Conf.*, 2007, pp. 2.A.1–1–2.A.1–10.

[2] A. R. I. Software, *ARINC Specification 653-1 Avionics application standard interface.*, 2003.

[3] J. Rushby, "Partitioning in avionics architectures: Requirements, mechanisms, and assurance," SRI Int., Technical Report, 1999.

[4] V. Muttillo, G. Valente, and L. Pomante, "Design space exploration for mixed-criticality embedded systems considering hypervisor-based sw partitions," in *2018 21st Euromicro Conf. on Digital System Design (DSD)*, 2018, pp. 740–744.

[5] N. Dejon, C. Gaber, and G. Grimaud, "From mmu to mpu: adaptation of the pip kernel to constrained devices," in *Proc. of the 3rd Int. Conf. on Internet of Things & Embedded Systems (IoTE 2022)*, Sidney, AU, December 2022.

[6] *ARM cortex-M23*. [Online]. Available: https://developer.arm.com/documentation/ddi0550/c/

[7] *AVR32UC microprocessor*. [Online]. Available: https://ww1.microchip.com/downloads/en/DeviceDoc/doc32002.pdf

[8] S. Pinto, H. Araujo, D. Oliveira, J. Martins, and A. Tavares, "Virtualization on trustzone-enabled microcontrollers? voilà!" in *2019 IEEE Real-Time and Embedded Technology and Applications Symposium (RTAS)*, 2019, pp. 293–304.

[9] "The risc-v instruction set manual volume ii: Privileged architecture." [Online]. Available: https://riscv.org/wp-content/uploads/2019/08/riscv-privileged-20190608-1.pdf

[10] J. Ma, T. Chong, L. Li, H. Liu, X. Zhang, L. Liu, and Y. Yuan, "Construction of risc-v lightweight trusted execution environment based on hardware extension," pp. 237–242, 2021.

[11] RISC-V TEE Task Group, *Secondary M-mode Trusted ConText Extension*. [Online]. Available: https://github.com/riscvarchive/riscv-tee/tree/386c5e81a69664099d5676786ef34e0f06688e46

[12] A. Cilardo and S. Mercogliano, "Flexible privilege management for microcontroller-class risc-v cores," *Microelectronics Reliability*, vol. 137, p. 114771, 2022.

[13] M. Hill, *TESx: Trusted Execution State*. [Online]. Available: https://lindemer.github.io/VEDLIoT/draft/tes.pdf

[14] G. D'Andrea, T. Di Mascio, and G. Valente, "Self-adaptive loop for cpss: is the dynamic partial reconfiguration profitable?" in *2019 8th Mediterranean Conf. on Embedded Computing (MECO)*, 2019, pp. 1–5.

Authorized licensed use limited to: University of L'Aquila. Downloaded on October 24,2025 at 10:25:44 UTC from IEEE Xplore. Restrictions apply.